



北京邮电大学
Beijing University of Posts and Telecommunications

北京邮电大学

计算机学院

Python 链家五大城市租房数据爬取与分析实验报告

姓名: 吴清柳

学号: 2020211597

班级: 2020211323

指导老师: 邵莹侠

课程名称: 算法设计与分析

December 31, 2022

Contents

1	概述	3
2	爬虫设计和数据爬取	3
2.1	行政区划和板块爬虫	3
2.1.1	爬取结果的持久化	3
2.2	小区爬虫	4
2.2.1	已爬标记和翻页	4
2.2.2	爬取结果的持久化	4
2.3	租房房源爬虫	4
2.3.1	已爬标记和翻页	5
2.3.2	爬取结果的持久化	5
2.4	部署在 scrapyd 上进行爬取	5
3	数据分析和处理	5
4	实验总结	6
5	附录	6

1 概述

在本实验中,选取北京,上海,广州,深圳和潍坊五个城市,借助 Scrapy 框架设计爬虫,部署在 Scrapyd 上运行爬取;爬取数据以定义的 item 结构保存,通过 pipeline 传输到使用 SQLite3 建立的数据库进行去重,断点续爬和持久化.使用 pandas 与 matplotlib 进行数据分析和可视化.

由于链家 100 页只有 3000 个房源,远小于房源总数,直接对租房房源进行爬取将遗漏大量房源信息.因此为了尽可能多的爬取房源,制作了三个爬虫.分别爬取板块(商圈, business area),小区 (community) 和租房房源 (rental).在爬取板块的同时,爬虫可以自动收集行政区划,小区指北京邮电大学家属小区这一级,租房房源指具体的出租房源信息.由于一个小区内不会有超过 3000 各租房房源,因此通过这一划分方式,可以尽可能的避免这一限制造成的爬取不充分.

由于实现了上述的较高程度的自动化,爬虫对城市的增加有极强的扩展性.如果需要添加新的城市,只需要添加新城市的租房 url 即可,而不需要手动添加待添加城市的具体行政区划等信息.

爬虫共计爬取商圈 800 个,其中北京 255 个,上海 172 个,广州 227 个,深圳 74 个,潍坊 72 个.小区 41262 个,其中上海 17211 个,北京 10213 个,广州 6575 个,深圳 4098 个,潍坊 3165 个.租房房源共 175570 个,其中北京 31095 个,上海 22998 个,广州 74387 个,深圳 37840 个,潍坊 9250 个.由于商圈,小区和房源的 url 中都有其唯一编码,故在插入 SQLite 的时候借助 unique 关键字对其进行去重.

2 爬虫设计和数据爬取

在本实验中,选取北京,上海,广州,深圳和潍坊五个城市,借助 Scrapy 框架设计爬虫,部署在 Scrapyd 上运行爬取;爬取数据以定义的 item 结构保存,通过 pipeline 传输到使用 SQLite3 建立的数据库进行去重,断点续爬和持久化.

爬虫设置方面,由于部署在 Scrapyd 上进行爬取,因此爬取时间不成为限制.为了避免可能出现的反爬措施,设置下载延迟为 610 秒,每个域名并行请求数为 2,每个 IP 并行请求数为 2,并使用了轮换的 User Agent.

由于链家一次只显示 100 页,只有 3000 个房源,远小于房源总数,直接对租房房源进行爬取将遗漏大量房源信息.因此为了尽可能多的爬取房源,制作了三个爬虫,分别爬取板块(商圈, business area),小区 (community) 和租房房源 (rental).在爬取板块的同时,爬虫可以自动收集行政区划,小区指北京邮电大学家属小区这一级,租房房源指具体的出租房源信息.由于一个小区内不会有超过 3000 各租房房源,因此通过这一划分方式,可以尽可能的避免这一限制造成的爬取不充分.对三个爬虫分别介绍如下.

2.1 行政区划和板块爬虫

这一爬虫是首先运行的爬虫,定义在“business_area_spider.py”中的 `class BusinessAreaSpider` 中.

其部署到 scrapyd 或者本地运行后,将以 start_requests 中 urls 中的 url 为起点,对每个城市的行政区,以及板块进行爬取.

2.1.1 爬取结果的持久化

爬取到的板块将保存在定义在“items.py”中的 `class BusinessAreaItem` 中,并传输给定义在“pipeline.py”中的 `class DownBusinessAreaUrlPipeline`,由该 pipeline 判断此 BusinessAreaItem 所在的城市,行政区,以及此板块是否分别存在与相应数据库表中.如果不存在,则插入.

BusinessAreaItem 定义如下.

```
class BusinessAreaItem(scrapy.Item):  
    # Business area
```

```
business_area_url = scrapy.Field()
business_area_name = scrapy.Field()
business_area_region = scrapy.Field()
business_area_city = scrapy.Field()
```

分别存储了板块的 url, 名称, 行政区和所在城市. 而其保存到的 SQL 中的板块表还有一个 accessbit 字段, 用于判断该板块是否已经被下面介绍的小区爬虫爬取过. 借助这一字段, 可以方便的实现断点续爬: 不会对已经爬取过的小区重复爬取, 而是对因为意外中断而没有来得及爬取的板块中小区进行爬取.

2.2 小区爬虫

这一爬虫在行政区划和板块爬虫爬取完毕后运行, 定义在“community_spider.py”中的 `class ComemunitySpider`. 在本地或者 scrapyd 上启动后, 将首先访问 SQLite 数据库, 获取没有访问的板块 url 和对应城市 url, 并对其开始并行爬取. 由于小区数量很多, 一页不能完全展示, 所以需要处理翻页问题; 此外, 为了避免对一个区块的重复爬取, 需要设置已爬标记.

2.2.1 已爬标记和翻页

每爬取完一页小区后, 需要借助当前页码和总页数进行判断. 当前页和总页数均存储在 `response.xpath("//div[@class='house-list-page-box']/@pagedata")` 中. 如果已经是最后一页小区, 则设置该小区所在的 business_area 的 accessbit 为 1, 表示这一区块已经被爬取过. 如果不是最后一页小区, 则对 response.url 进行处理, 提取出当前板块的主 url, 并链接 `f"pg{next_page}/"` 字段, 作为下一页 url 进行爬取.

2.2.2 爬取结果的持久化

爬取结果保存在 `class CommunityItem` 中, 传输给 `class DownCommunityInfoPipeline`, 由其根据小区链接 `item["community_url"]` 是否已在数据库中来判断该结果是否重复. 如果不重复, 则插入数据库. `CommunityItem` 定义如下.

```
class CommunityItem(scrapy.Item):
    # Community
    community_url = scrapy.Field()
    community_name = scrapy.Field()
    community_region = scrapy.Field()
    community_city = scrapy.Field()
    community_business_area = scrapy.Field()
    community_rent_url = scrapy.Field()
```

分别存储小区链接, 小区名, 小区所在行政区, 小区所在城市, 小区所在板块和小区租房链接. 其中如果小区内没有房屋出租, 则租房链接为 'None', 避免数据库中出现 null 值.

2.3 租房房源爬虫

小区信息爬取完毕后, 在小区信息的基础上进行租房房源信息的爬取. 租房房源爬虫为“rent_spider.py”中的 `class RentSpider(scrapy.Spider)`, 首先从数据库中读取租房链接 `community_rent_url` 非 'None', 且访问位 `community_accessbit` 为 0 的小区列表, 再对这些小区进行租房房源的并行爬取. 由于单小区内

房源也会有一页不能完全展示的问题, 因此也需要处理翻页问题. 链家的租房房源不同页的 url 设计比较奇怪, 给翻页问题的处理带来了一些困扰.

2.3.1 已爬标记和翻页

与爬取板块内小区不同的是, 小区内会出现没有房源正在出租的情况. 通过仔细观察, 注意到字段 `response.xpath("//span[@class='q']/text())` 记录了当前小区内房源数量. 如果当前小区内租房房源数量为 0, 则设置当前小区的访问位 `community_accessbit` 为 2, 跳过当前小区.

否则, 每爬取完小区内的一页租房房源后, 需要借助当前页码和总页数进行判断. 当前页和总页数均存储在 `response.xpath("//div[@class='content__pg']")` 中. 如果已经是最后一页小区, 则设置该页所在小区的 `community_accessbit` 为 1, 结束对该小区的访问.

2.3.2 爬取结果的持久化

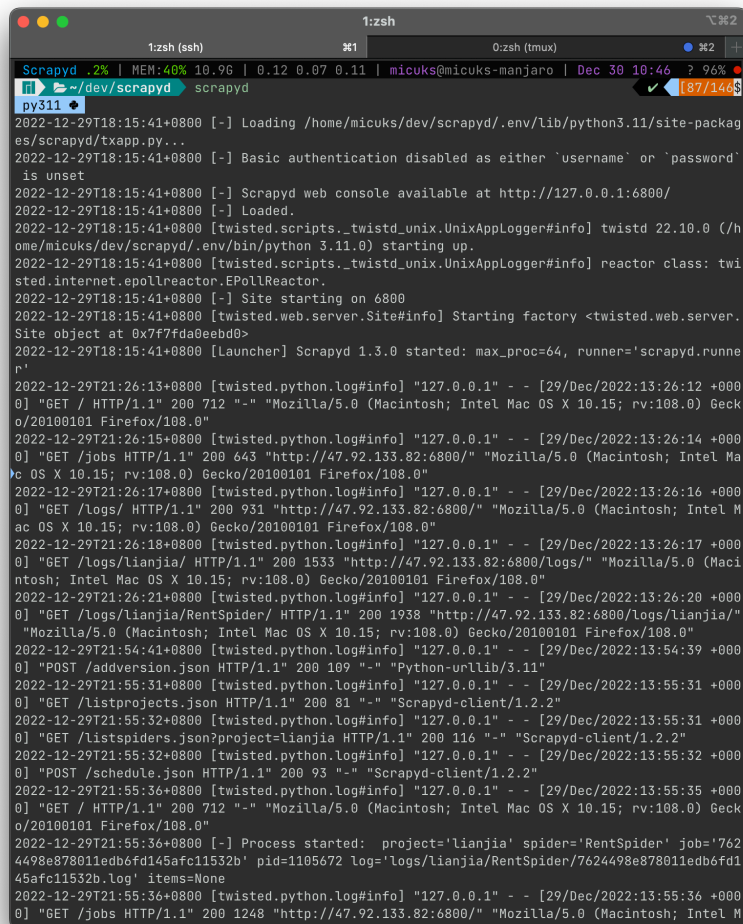
爬取结果保存在 `class RentalItem` 中, 传输给 `class DownRentalInfoPipeline`, 由其根据房源 url 判断是否已经在 SQLite 数据库中的 rental 表中. 如果不存在, 则插入. `RentalItem` 定义如下:

```
class RentalItem(scrapy.Item):
    # Rental info
    rental_name = scrapy.Field()
    rental_url = scrapy.Field()
    rental_city=scrapy.Field()
    rental_region = scrapy.Field()
    rental_business_area = scrapy.Field()
    rental_community_url=scrapy.Field()
    rental_community = scrapy.Field()
    rental_area = scrapy.Field()
    rental_lighting = scrapy.Field()
    rental_rooms = scrapy.Field()
    rental_liverooms = scrapy.Field()
    rental_bathrooms = scrapy.Field()
    rental_price = scrapy.Field()
    rental_timestamp = scrapy.Field()
```

2.4 部署在 scrapyd 上进行爬取

完成上述过程后, 为了让爬虫能够稳定运行, 在服务器上安装 scrapyd, 部署在端口 6800 并转发端口. 在本地使用 scrapyd-client 将爬虫部署到服务器的 scrapyd, 并依次运行 `BusinessAreaSpider`, `CommunitySpider` 和 `RentSpider` 进行爬取.

3 数据分析和处理



```
Scrapy .2% | MEM:40% 10.96 | 0.12 0.07 0.11 | micuks@micuks-manjaro | Dec 30 10:46 ? 96%
~/dev/scrapyd scrapyd
py311
2022-12-29T18:15:41+0800 [-] Loading /home/micuks/dev/scrapyd/.env/lib/python3.11/site-packag
es/scrapyd/txapp.py...
2022-12-29T18:15:41+0800 [-] Basic authentication disabled as either 'username' or 'password'
is unset
2022-12-29T18:15:41+0800 [-] Scrapy web console available at http://127.0.0.1:6800/
2022-12-29T18:15:41+0800 [-] Loaded.
2022-12-29T18:15:41+0800 [twisted.scripts._twistd_unix.UnixAppLogger#info] twisted 22.10.0 (/h
ome/micuks/dev/scrapyd/.env/bin/python 3.11.0) starting up.
2022-12-29T18:15:41+0800 [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twi
sted.internet.epollreactor.EPollReactor.
2022-12-29T18:15:41+0800 [-] Site starting on 6800
2022-12-29T18:15:41+0800 [twisted.web.server.Site#info] Starting factory <twisted.web.server.
Site object at 0x7f7fda0e8bd0>
2022-12-29T18:15:41+0800 [Launcher] Scrapy 1.3.0 started: max_proc=64, runner='scrapyd.runne
r'
2022-12-29T21:26:13+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:12 +000
0] "GET / HTTP/1.1" 200 712 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Geck
o/20100101 Firefox/108.0"
2022-12-29T21:26:15+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:14 +000
0] "GET /jobs HTTP/1.1" 200 643 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel Ma
c OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:17+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:16 +000
0] "GET /logs HTTP/1.1" 200 931 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel M
ac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:18+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:17 +000
0] "GET /logs/lianjia/ HTTP/1.1" 200 1533 "http://47.92.133.82:6800/logs/" "Mozilla/5.0 (Maci
ntosh; Intel Mac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:26:21+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:26:20 +000
0] "GET /logs/lianjia/RentSpider/ HTTP/1.1" 200 1938 "http://47.92.133.82:6800/logs/lianjia/"
"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Gecko/20100101 Firefox/108.0"
2022-12-29T21:54:41+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:54:39 +000
0] "POST /addversion.json HTTP/1.1" 200 109 "-" "Python-urllib/3.11"
2022-12-29T21:55:31+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:31 +000
0] "GET /listprojects.json HTTP/1.1" 200 81 "-" "Scrapy-client/1.2.2"
2022-12-29T21:55:32+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:31 +000
0] "GET /listspiders.json?project=lianjia HTTP/1.1" 200 116 "-" "Scrapy-client/1.2.2"
2022-12-29T21:55:32+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:32 +000
0] "POST /schedule.json HTTP/1.1" 200 93 "-" "Scrapy-client/1.2.2"
2022-12-29T21:55:36+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:35 +000
0] "GET / HTTP/1.1" 200 712 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Geck
o/20100101 Firefox/108.0"
2022-12-29T21:55:36+0800 [-] Process started: project='lianjia' spider='RentSpider' job='762
4498e878011edb6fd145afc11532b' pid=1105672 log='logs/lianjia/RentSpider/7624498e878011edb6fd1
45afc11532b.log' items=None
2022-12-29T21:55:36+0800 [twisted.python.log#info] "127.0.0.1" - - [29/Dec/2022:13:55:36 +000
0] "GET /jobs HTTP/1.1" 200 1248 "http://47.92.133.82:6800/" "Mozilla/5.0 (Macintosh; Intel M
```

Figure 1: 运行在 Scrapy 上的爬虫

4 实验总结

通过这次实验, 我体验了从零开始设计爬虫, 爬取数据到数据分析处理的全部过程, 体会到了 python 的方便强大.

在爬虫部署的工具方面, 使用了 scrapy 工具, 方便的实现了爬虫的服务器端自动化爬取.

但是, 由于时间有限, 且感染了奥密克戎, 深受发烧困扰, 没能来得及进行进一步的优化. 例如, 将使用 jupyter notebook 书写的数据分析部分与前面 latex 书写的爬虫介绍部分更好的组合在一起. 也没有实现更多的优化内容, 例如 IP 池等, 比较遗憾.

总的来说, 这次实验在爬虫, 数据分析和 Python 之内及之外都学到了许多知识, 收获颇丰.

5 附录

代码仓库: [Lianjia](#)